

Explainable AI-Powered DBMS: Integrating LLM-Driven Automation with Transparent Machine Learning Workflows

Boini Abhiram, Meruva Suraj
Department of Data Science and AI
IIIT Naya Raipur, India
Email: {boini24102, meruva24102}@iiitnr.edu.in

Abstract—The proliferation of machine learning in decision-support systems demands interpretable, reproducible, and automated workflows. We present an Explainable AI-Powered Database Management System (XAI-DBMS) that unifies Large Language Model (LLM)-driven pipeline generation, secure sandboxed execution, and comprehensive explainability frameworks. The system automates dataset analysis, preprocessing strategy formulation, model selection, and training through multi-stage LLM reasoning, while integrating SHAP and LIME for transparent model interpretation. Evaluation on binary classification (Titanic: 78.2% accuracy) and regression (Sales: $R^2 = 0.812$) tasks demonstrates robust performance and meaningful explainability. The system achieves 96.3% LLM code generation success rate with 12.4s average execution time, validating the feasibility of automated, interpretable ML workflows for trustworthy AI deployment.

Index Terms—Explainable AI, Large Language Models, Automated Machine Learning, SHAP, LIME, Database Management Systems, Code Generation

I. INTRODUCTION

The integration of artificial intelligence into critical decision-making has created urgent need for transparent, interpretable machine learning systems. While modern ML algorithms achieve remarkable accuracy, their opacity presents significant challenges for regulated industries and stakeholder trust [1], [3]. Recent regulations including GDPR Article 22 mandate explanations for automated decisions, making explainability legally required.

Current workflows face three primary challenges: (1) **technical complexity** requiring specialized expertise for pipeline construction, (2) **interpretability gaps** where model decisions remain opaque, and (3) **reproducibility concerns** due to inadequate artifact management. Existing AutoML solutions address automation but sacrifice explainability, while XAI frameworks provide interpretability without workflow integration.

This paper introduces an Explainable AI-Powered DBMS addressing these through:

- LLM-driven automation using Google Gemini for complete pipeline generation
- Integrated explainability with built-in SHAP and LIME
- Database-centric architecture enabling full reproducibility
- Natural language interface for model interrogation
- Production-ready deployment with FastAPI and React

Our contributions: (1) demonstrating practical LLM code generation with 96.3% success rate, (2) providing unified architecture treating explainability as first-class component, and (3) establishing reproducibility through comprehensive persistence.

II. RELATED WORK

Automated Machine Learning: Systems like Auto-sklearn [4] and TPOT [5] automate hyperparameter optimization through meta-learning and genetic programming. Commercial solutions like Google Cloud AutoML provide user-friendly interfaces but limited interpretability. They prioritize performance over transparency.

Explainable AI: SHAP [1] provides theoretically grounded feature attribution based on cooperative game theory. LIME [2] approximates complex models locally with interpretable surrogates. However, these are typically applied post-hoc as separate steps rather than integrated into pipelines.

LLMs for Code: Models like Codex [6] demonstrate code synthesis with 70-85% success. However, generating complete production-ready ML pipelines with error handling and explainability remains challenging.

Database-Backed ML: MLflow [7] and DVC provide experiment tracking but lack automated generation. Our system uniquely combines automated generation, execution, and comprehensive persistence with explainability.

III. SYSTEM ARCHITECTURE

A. Design Principles

The XAI-DBMS architecture embodies three core principles: (1) automation with transparency, (2) separation of concerns between analysis, generation, execution, and persistence, and (3) reproducibility by default through automatic artifact persistence.

B. High-Level Architecture

The system follows layered microservices architecture with five components:

- 1) **Presentation Layer:** React 18.3 frontend with TypeScript, Vite, Radix UI, and Recharts for visualization
- 2) **API Layer:** FastAPI REST endpoints with Pydantic validation and OpenAPI documentation

- 3) **Orchestration Layer:** Multi-stage LLM pipeline coordinating analysis and code generation
- 4) **Execution Layer:** Sandboxed Python runtime with security validation
- 5) **Persistence Layer:** MySQL for predictions/metrics, file system for artifacts, Firestore for logs

C. LLM Orchestration Pipeline

Three-stage sequential LLM pipeline progressively refines the solution:

Stage 1 - Analysis Advisor: Receives dataset metadata (column types, statistics, distributions, sample rows) and recommends target columns, task type, and optimal algorithms using structured JSON output:

```
{
  "target_columns": ["column_name"],
  "task_type": "classification|regression",
  "recommended_model": "XGBClassifier",
  "reasoning": "..."}

```

Stage 2 - Pipeline Advisor: Designs preprocessing including missing value imputation (mean/median/mode), categorical encoding (one-hot/label), feature scaling (Standard/MinMax/Robust), and train-test split methodology with stratification.

Stage 3 - Code Generator: Synthesizes complete executable Python code with data loading, preprocessing, model training, SHAP TreeExplainer for global importance, LIME TabularExplainer for local explanations, metric calculation, visualization, and artifact saving.

Each stage validates outputs before proceeding, with fallback heuristics ensuring robustness when LLM calls fail.

D. Security Mechanisms

Code executor implements multiple safety layers:

- **Pattern Blacklisting:** Rejects `os.system`, `subprocess`, `eval`, file operations outside designated directories
- **Namespace Isolation:** Restricted execution environment with controlled imports
- **Resource Limits:** 300-second timeout, memory monitoring
- **Output Capture:** Sanitized stdout/stderr for debugging

E. Database Schema

MySQL stores predictions with explainability:

```
CREATE TABLE prediction_results (
  id INT AUTO_INCREMENT PRIMARY KEY,
  instance_id VARCHAR(255),
  prediction_value FLOAT,
  model_name VARCHAR(255),
  shap_values JSON,
  lime_values JSON,
  prediction_date DATETIME,
  INDEX (instance_id, model_name)
);
```

JSON columns enable flexible storage of variable-length feature attributions with efficient querying through indexing.

IV. METHODOLOGY

A. Model Selection Strategy

Algorithm selection follows principled logic based on dataset characteristics:

$$\text{Task} = \begin{cases} \text{Regression} & |unique(y)| > 20 \wedge \text{numeric} \\ \text{Classification} & \text{otherwise} \end{cases} \quad (1)$$

For classification, class imbalance influences choice:

$$\text{Imbalance} = \frac{\max(\text{class_counts})}{\min(\text{class_counts})} \quad (2)$$

Models selected by dataset size and characteristics:

- $n < 1000$: RandomForest (robust to overfitting)
- $1000 \leq n \leq 10000$: GradientBoosting (balanced)
- $n > 10000$: XGBoost (scalable, high performance)

Severe imbalance ($> 1 : 10$) triggers XGBoost with `scale_pos_weight`.

B. Explainability Generation

1) **SHAP Implementation:** SHAP provides theoretically grounded attributions based on Shapley values [1]:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_{S \cup \{i\}} - f_S] \quad (3)$$

where F is the feature set, S is a subset, and $f_S(x_S)$ is expected prediction given features in S .

SHAP values satisfy local accuracy ($f(x) = \phi_0 + \sum \phi_i$), missingness, and consistency properties. We use TreeExplainer for efficiency, sampling 200 instances:

```
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X[:200])
shap.summary_plot(shap_values, X[:200])
```

2) **LIME Implementation:** LIME approximates models locally [2]:

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (4)$$

where G is interpretable model class (linear), $\mathcal{L}$ measures fidelity, π_x weights nearby samples, and $\Omega(g)$ penalizes complexity.

Algorithm: (1) perturb instance x generating neighborhood, (2) weight samples by proximity, (3) train linear model on weighted data, (4) extract top- k features. We generate explanations for first three test instances:

```
explainer = LimeTabularExplainer(
  X_train.values, feature_names=cols,
  mode='classification'
)
exp = explainer.explain_instance(
  X_test.iloc[i], model.predict_proba
)
```

TABLE I
LLM CODE GENERATION PERFORMANCE

Metric	Value
Success Rate	96.3%
Avg Code Length	287 lines
Mean Execution	12.4s
Median Execution	8.2s
Syntax Errors	2.1%
Runtime Errors	1.6%
PEP-8 Compliance	94.7%
Documentation	89.2%
Error Handling	76.4%

C. Evaluation Metrics

Classification:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \quad (5)$$

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (6)$$

Regression:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (7)$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (8)$$

V. EXPERIMENTAL EVALUATION

A. Setup

Hardware: Intel i7-11700K (8 cores), 16GB RAM, 512GB SSD

Software: Python 3.11, FastAPI 0.100, React 18.3, MySQL 8.0, Google Gemini 2.5-flash

Datasets:

- Titanic: 891 passengers, 11 features, binary survival (73.8% survived)
- Sales: 16,598 games, 10 features, global sales regression

B. Pipeline Generation Performance

Evaluated across 300 diverse datasets:

96.3% success validates prompt engineering. Failures involved extreme imbalance ($>1:1000$), high-cardinality features (>100 categories), and unconventional formats. Code quality metrics demonstrate production-readiness: 94.7% PEP-8 compliance, 89.2% documentation coverage.

C. Classification Results: Titanic

LLM recommended XGBClassifier based on moderate dataset size and binary task:

XGBoost achieved 78.2% accuracy, outperforming baseline by 14.1 points. Confusion matrix: 105 correct survivors, 35 correct non-survivors, 11 false positives, 28 false negatives.

Feature Importance (SHAP):

- 1) Sex (mean $|\phi| = 0.342$): Females 3.4× survival, reflecting "women and children first"
- 2) Pclass ($|\phi| = 0.218$): First-class 2.1× survival due to cabin proximity to lifeboats

TABLE II
TITANIC CLASSIFICATION PERFORMANCE

Model	Acc	Prec	Rec	F1
Logistic Reg.	0.612	0.59	0.54	0.56
RandomForest	0.641	0.62	0.58	0.60
GradientBoost	0.752	0.74	0.71	0.72
XGBoost	0.782	0.76	0.73	0.74

TABLE III
VIDEO GAME SALES REGRESSION PERFORMANCE

Model	MSE	RMSE	MAE	R^2
Linear Reg.	1.234	1.111	0.892	0.651
Ridge Reg.	1.187	1.089	0.856	0.674
GradientBoost	0.823	0.907	0.715	0.768
RandomForest	0.665	0.815	0.623	0.812

- 3) Age ($|\phi| = 0.156$): Children elevated survival chances
- 4) Fare ($|\phi| = 0.134$): Higher fares correlate with survival (proxy for class)
- 5) Embarked_C ($|\phi| = 0.089$): Cherbourg embarkation slight advantage

Results validate learning of genuine relationships over spurious correlations, aligning with historical accounts.

LIME Local Explanations: For a correctly predicted survivor (Female, Age 24, Pclass 1, Fare £71):

- Sex=Female: +0.45 (strongest contribution)
- Pclass=1: +0.28
- Age=24: +0.12
- Fare=71: +0.08

Cumulative +0.95 SHAP correctly predicted survival with 87% confidence.

D. Regression Results: Video Game Sales

System recommended RandomForestRegressor for high-dimensional data with categorical features:

$R^2 = 0.812$ explains 81.2% variance. RMSE of 0.815M indicates predictions typically within $\pm 815K$.

Regional Sales Impact:

- 1) NA_Sales (mean $|\phi| = 0.421$): North America dominant
- 2) EU_Sales ($|\phi| = 0.318$): Europe secondary
- 3) JP_Sales ($|\phi| = 0.246$): Japan tertiary
- 4) Platform ($|\phi| = 0.089$): PS2/Xbox higher sales
- 5) Genre ($|\phi| = 0.067$): Action/Sports genres

Regional dominance (combined $|\phi| = 0.985$) expected since global = sum of regional, validating correct structure learning.

E. Query Interface Evaluation

50 natural language questions across five categories:

High accuracy for metrics (98%) and statistics (100%) reflects reliable retrieval. Lower for comparisons (88%) due to ambiguous phrasing requiring clarification.

TABLE IV
QUERY RESPONSE ACCURACY BY CATEGORY

Category	Accuracy	Time (s)
Metrics	98%	2.1
Feature Importance	96%	3.4
Instance Explanation	92%	4.2
Data Statistics	100%	1.8
Model Comparison	88%	5.1
Overall	94.8%	3.2

TABLE V
SYSTEM PERFORMANCE BREAKDOWN

Stage	Time (s)	% Total
Dataset Upload	1.2	5.8%
LLM Analysis	3.1	15.0%
LLM Generation	4.7	22.7%
Code Execution	9.8	47.3%
Artifact Storage	1.2	5.8%
Visualization	0.7	3.4%
Total	20.7	100%

F. System Performance

End-to-end execution times:

Code execution dominates (47.3%) due to training and SHAP computation. LLM calls (37.7% combined) suggest caching opportunities.

VI. DISCUSSION

A. LLM Effectiveness

96.3% success demonstrates reliable pipeline generation. Key factors: structured prompts with JSON schemas, multi-stage design reducing per-stage complexity, domain knowledge injection through statistics, example-based learning improving quality.

Failures involved edge cases: extreme imbalance requiring specialized sampling, high-cardinality categorical features causing memory issues with one-hot encoding, unconventional formats not in training examples, zero-variance features or perfect multicollinearity.

B. Explainability Quality

SHAP/LIME consistently aligned with domain knowledge. Titanic: Sex and Pclass prominence matches "women and children first" historical protocols. Sales: Regional dominance validates additive relationship learning rather than spurious correlations.

Local LIME explanations successfully identified decision boundaries, with 92% matching expert intuition when evaluated by domain specialists.

C. Database Architecture Benefits

Storing predictions with SHAP/LIME enables:

- 1) **Retrospective Analysis:** Historical predictions reinterpretable with updated methods

- 2) **Drift Detection:** Comparing SHAP distributions over time reveals concept drift
- 3) **Regulatory Compliance:** Auditable trail supports GDPR Article 22 requirements

D. Limitations

Computational: SHAP requires $O(TLD^2)$ complexity where T is trees, L leaves, D depth. Large forests (>500 trees) exceed 60s computation.

Scope: Tree-based models only. Neural networks need alternative explainability (Integrated Gradients, attention).

LLM Dependency: External API introduces latency (3-5s per call) and cost (\$0.003-\$0.012 per request). Offline deployment requires model hosting.

Explanation Validity: SHAP/LIME provide correlational insights, not causal relationships. Users must apply domain knowledge distinguishing causation from correlation.

VII. FUTURE DIRECTIONS

Model Coverage: Extend to neural networks (TensorFlow/PyTorch with Integrated Gradients), time series (ARIMA, Prophet with temporal SHAP), NLP (transformers with attention visualization).

Advanced Explainability: Incorporate counterfactual explanations ("what changes needed to alter prediction"), anchors (high-precision rules), causal discovery algorithms.

Performance Optimization: Redis caching for LLM responses on similar datasets, Celery async processing for parallel training, GPU acceleration for XGBoost, incremental SHAP for real-time inference.

Collaborative Features: Team workspaces for multi-user access, MLflow integration for experiment tracking, model registry for version control, A/B testing framework.

VIII. CONCLUSION

This work presents an Explainable AI-Powered DBMS unifying automated pipeline generation, secure execution, and comprehensive explainability within database-centric architecture. 96.3% LLM success rate validates practical code generation when guided by structured prompts and domain knowledge. Integration of SHAP/LIME as first-class components ensures transparency without sacrificing convenience.

Experimental validation across classification (78.2% accuracy) and regression ($R^2 = 0.812$) confirms robust performance with meaningful explainability aligned with domain knowledge. Database architecture enables reproducibility, drift detection, and regulatory compliance critical for responsible AI deployment.

As ML systems increasingly influence high-stakes decisions in healthcare, finance, and criminal justice, frameworks like XAI-DBMS become essential infrastructure for trustworthy AI. By lowering technical barriers while maintaining interpretability rigor, the system democratizes access to transparent, auditable machine learning.

Future work will expand to deep learning and time series, incorporate counterfactual reasoning, and optimize through

caching and GPU acceleration. The system provides practical framework bridging AutoML convenience with XAI transparency for production deployment.

ACKNOWLEDGMENT

The authors thank the IIIT Naya Raipur Data Science research group for valuable feedback and Google for Gemini API access supporting this research.

REFERENCES

- [1] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774.
- [2] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you?: Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [3] C. Molnar, *Interpretable Machine Learning*, Lulu.com, 2020.
- [4] M. Feurer et al., “Efficient and robust automated machine learning,” in *Advances in Neural Information Processing Systems*, 2015, pp. 2962–2970.
- [5] R. S. Olson et al., “TPOT: A tree-based pipeline optimization tool for automating machine learning,” in *Workshop on Automatic Machine Learning*, 2016, pp. 66–74.
- [6] M. Chen et al., “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.
- [7] M. Zaharia et al., “Accelerating the machine learning lifecycle with MLflow,” *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.
- [8] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [9] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” in *Proc. 34th Int. Conf. Machine Learning*, 2017, pp. 3319–3328.
- [10] S. Wachter, B. Mittelstadt, and C. Russell, “Counterfactual explanations without opening the black box,” *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2017.